Beyond Chronological Splits: A Deployment-Causality and Falsifiability Contract for Learning-Assisted Satellite Communication Software

Anonymous Submission

***Abstract*—Chronological train/validation/test splitting is a common and necessary practice for temporal learning pipelines, but ordering is only one property a deployment claim rests on. A pipeline can satisfy it exactly and still be uninterpretable, because ordering constrains neither whether a past-dated quantity was *available* at the decision instant, nor which rows *exist*, nor what a learner's hidden state has observed, nor whether the unit over which intervals are computed is exchangeable. Satellite software makes these concrete: an orbital element set carries an epoch and a separate publication time, a schedule is meaningful only inside predicted-visible geometry, retrospective labels arrive when a catalogue chooses to publish, and samples from one pass are repeated measures. We formalise six protected objects—decision time, feature availability, row membership, label closure, state channels and statistical units—as an executable contract of nineteen rules in four mechanically checkable layers, and release *Orbit-Evidence*, a dependency-light implementation. On a curated suite of seventeen fault classes over two pipelines and three deterministic environments, an implemented chronological baseline detects the $2/17$ classes that are ordering properties and the contract detects $17/17$; clean reference paths are accepted, the statistical-unit rule's measured false-halt rate is 0.042 against a nominal 0.05, and the sweep runs in under 2 s. This is *represented-fault* regression coverage; it does not establish completeness, nor generalisation to faults the suite lacks. We make no radio-frequency, packet-level or learned-method performance claim.**

***Index Terms*—experiment validity, data leakage, reproducibility, non-terrestrial networks, learning-assisted communication, regression testing**

I. Introduction

Learning components are increasingly proposed as assistants to physical models in satellite and non-terrestrial communication software: correcting a propagated orbital prediction, adapting a link parameter, or deciding whether a learned branch should override a model-based default [1]. Because such systems are temporal, evaluation is organised as a chronological split—train on the past, validate on the near future, test on the far future. That is a common and necessary practice for temporal learning pipelines, and the conditions under which ordered splitting is valid are well established [2].

Our claim is that it is not sufficient, and that the gap is one of kind rather than degree. A chronological split constrains the *order* of quantities already in a dataset, and is silent on four further questions. *Availability*: a quantity may be past-dated and still unpublished, so no system could have held it. *Membership*: which rows exist can itself be decided by later data, in both directions, and a split cannot examine rows never created. *Hidden state*: a split governs rows, whereas information travels through a scaler, a coefficient vector, a tracker's latent state, selection metadata, or an admission bit—none of which is a column. *Statistical units*: a split says nothing about whether the unit over which uncertainty is computed is exchangeable.

A. Why the satellite domain makes this concrete

These gaps are abstract in general and specific here. Six properties of satellite communication software give each protected object an operational meaning, and are why we develop the contract in this domain rather than as generic tooling.

(i) Two clocks. An orbital element set has an *element epoch*—the instant its state vector describes—and a separate *publication time* at which a catalogue released it [3]; these differ by hours. A feature formed from two element epochs is a difference of two past timestamps, so it passes every ordering test, and is still unavailable if either element was published after the decision instant. *(ii) A decision instant that is a real endpoint.* Open-loop decisions use the state an endpoint *holds*, not the best that exists; staleness is normal, so availability is first-class rather than an edge case. *(iii) Schedules live inside predicted geometry.* An opportunity exists only where the link is predicted visible above an elevation mask, so sampling a uniform grid and filtering afterwards yields rows no scheduler would emit. *(iv) Labels arrive retrospectively, from a publisher.* A better-determined later element set can serve as a label, but whether and when it appears is the catalogue's decision; later data may set a label's value, uncertainty or closure time, and must never decide whether a row existed. *(v) Passes are repeated measures.* Samples within one pass share geometry and one propagation state, so the pass or deployment episode is often the appropriate physical unit. *(vi) State survives a nominal freeze.* Stateful link adaptation and residual trackers update at their own refresh events, which can fall after a model

is declared frozen—so the learner keeps observing outcomes even when the feature tensor is verifiably clean.

We therefore treat experimental validity as an *executable contract* rather than a protocol description: each rule protects a named object, states a violation, and is enforced by a check reporting the rule it broke. Our thesis: Orbit-Evidence turns deployment-time availability, row membership, model-state and statistical-unit assumptions into executable CI checks for satellite communication experiments, and a curated regression suite demonstrates those checks on seventeen known fault classes that chronological ordering alone is not designed to detect.

Contributions.

1) **A deployment-causality threat model and contract.** Six protected objects and the violations each admits, as nineteen executable rules in four mechanically checkable layers: availability and closure (L1), physical and scheduling validity (L2), model-state causality (L3), statistical independence and reproducibility (L4). Table I.
2) **Orbit-Evidence, an implementation.** A dependency-light toolkit (833 lines across four modules, `numpy` only) providing visible-pass scheduling, a freeze-then-label row registry, reference-ensemble labelling with published uncertainty, canaries over six state channels—feature tensor, scaler, model coefficients, tracker state, selected-model metadata, gate bit—and seed, provenance and statistical-unit controls.
3) **A curated fault-injection regression evaluation.** Seventeen fault classes in two minimal pipelines and three deterministic environments: an implemented chronological protocol check detects $2/17$, the contract detects $17/17$, clean reference paths are accepted, the sweep is deterministic and runs in under 2 s, and two case studies show defects chronology does not constrain. We report this as *represented-fault* coverage—evidence that the rules catch the violations the suite contains, not sensitivity to faults it lacks (§VI).

We claim nothing about radio-frequency or packet-level performance. No learned communication method is shown here to work or to fail; the object of study is the experiment, not the model.

Availability. The toolkit, fixtures, fault suite and summary artifact accompany this submission anonymously; `make gate` regenerates the evaluation, rebuilds this paper and refuses to build if any number here disagrees with the artifact.

II. Threat Model and Experiment Contract

A. Decision time and the six protected objects

Let t_d be the *decision instant*: the moment a deployed system must act using only state it already holds—for an open-loop satellite endpoint, when a transmission is committed from the element set on board. A deployment claim is interpretable only if every quantity entering the decision was obtainable before t_d. Six objects are therefore *protected*; Fig. 1 shows their relationship in time.

Feature availability and the *availability clock* (§I-A i). A feature must be computable from state available at t_d. The adversarial case is not a future-dated feature—ordering catches that—but one built from two past timestamps, one belonging to an item published after t_d. Descriptive and publication timestamps are distinct clocks and selection must use publication; conflating them admits data the system could not have held, and the error is silent because both values lie in the past.

Row membership (§I-A iv). Later information may change a label's status, value, uncertainty or closure time, but never whether a row exists—violations run both ways: rows dropped when a later reference fails to appear, rows created when a schedule's extent comes from the data on hand rather than a declaration.

Label closure. A retrospective label becomes usable only at a closure time t_c. Its use before t_c is an ordering violation and *is* caught by a split—one of the two classes in our suite for which that holds, drawn dotted in Fig. 1 for contrast. The harder failure is when the *probability* of a label existing depends on the covariate under study: a publication outage makes the held element stale and simultaneously removes the elements needed to label that staleness, a missingness property no split addresses.

State channels (§I-A vi). Beyond the feature tensor, information enters through the scaler, model coefficients, a tracker's latent state, selected-model metadata, or the gate bit; a canary scoped to features misses five of six. *Statistical units* (§I-A v). Replicates of one physical event are not independent observations, and aggregating at some level does not establish that the level is exchangeable.

B. Four layers

L1 protects availability, the clock distinction, closure and membership. L2 protects physical validity: a schedule must lie inside geometry that can carry the link (§I-A iii), the target between a resolution floor and a plausibility ceiling, declared relations must match implementations, hidden state must not diverge without bound. L3 protects model-state causality through mutation canaries and a negative control. L4 protects fold chronology, paired randomness, repeated-measure aggregation, seed hygiene and provenance completeness.

Two design commitments make the contract usable. Every violation raises an exception carrying its *rule identifier*, so a finding names what it broke rather than surfacing a bare assertion failure. And a detector is trusted only once demonstrated to fail on a deliberately broken fixture: a check that cannot go red is not evidence. Table I separates rules with such a fixture from those exercised only on the clean path.

C. Relation to existing practice

Leakage taxonomies catalogue how information about a target reaches a model illegitimately [4], and surveys document its prevalence in machine-learning-based science [5]. Our threat model is narrower and operational: it asks not whether a quantity is legitimate but whether the deployed system could

have *held* it, turning a modelling concern into a scheduling and provenance one. Time-series work establishes when ordered splitting is valid [2]; we address what it does not cover. The hidden-state channels we guard instance the configuration and state coupling identified as technical debt in production learning systems [6]. Our method is fault injection in the tradition of mutation testing [7] applied to a protocol rather than program logic; manifests follow the discipline urged for datasets [8], and element conventions are standard [3].

III. Orbit-Evidence Toolkit

Visible-pass scheduler. Transmissions are *generated* from intervals predicted by the state the system holds, located by a coarse scan and refined by bisection to both threshold crossings—each to the above-threshold side of the bracket, so no scheduled sample falls below the mask through refinement error, at the cost of a reported extent that is a slight underestimate. Elevation uses the geodetic normal, not the geocentric radius. The visibility *criterion* (mask, minimum pass duration) is declared, and so are the *solver* settings locating its boundaries: coarse step, bisection tolerance and iteration cap all enter the provenance manifest, since the schedule is a numerical solution. The coarse step may not exceed the minimum pass duration—a shortest-admissible pass could otherwise hide between grid points—and a regression test sweeps the declared range of all three settings.

Freeze-then-label registry. Rows are assembled and hashed before any label source is consulted, against an *absolute declared* window—deriving either bound from the data on hand makes the schedule a property of the download. The falsifiable form compares registries built from full and truncated data under the same window.

Reference ensemble. Where a label must be assembled from several later observations, the toolkit canonicalises them deterministically, takes the median, and publishes a median-absolute-deviation uncertainty and a closure time. The row status is *coverage-only and diagnostic*: a function of member count and mutual spread against a declared ceiling, never of the labelled value or a physics baseline. That is a corrected defect: an earlier version compared the uncertainty to the residual, making the annotation a function of the quantity under study and biasing every completeness rate computed from it. The status never creates or deletes a row—large-spread rows are marked and *retained*, since dropping them selects on the outcome—and a split-half diagnostic is exposed because a mutual-deviation statistic cannot see an error shared by all members, so the published uncertainty is a lower bound.

A. Two rules that inspect a relation between runs

Provenance completeness (L4.6) cannot be verified by examining a manifest, which is silent about what it omits—but it is verifiable differentially. Over input variations $\{c_i\}$ producing output digests o_i and manifest hashes h_i, any pair with $o_i \neq o_j$ and $h_i = h_j$ proves the manifest incomplete, since two runs differing in output must differ in provenance. The converse is not an error: identical outputs under differing manifests is merely redundant. That asymmetry makes the rule checkable without enumerating every input.

Statistical unit selection (L4.7) asks not whether replicates were aggregated but whether the level they were aggregated *to* is exchangeable. Given values y, unit identifiers u and the next coarser grouping g (for a satellite experiment: samples aggregated to a pass, grouped by deployment episode), we aggregate to $\bar{y}_u$ and measure its intraclass correlation *within* g. Residual correlation there means the chosen unit shares state with its neighbours, so intervals over it are optimistic.

Estimator and reference distribution are both load-bearing. The estimator is the unbiased one-way random-effects $\mathrm{ICC}(1)$ from variance components, $\hat{\rho} = (\mathrm{MS_B} - \mathrm{MS_W})/(\mathrm{MS_B} + (\bar{m} - 1)\mathrm{MS_W})$ [9], whose expectation under independence is 0 at every group size; a raw between-over-total scatter ratio has null expectation $\approx 1/\bar{m}$, so any fixed threshold below that halts on correct designs. The decision is referenced to a *permutation null*: unit values are permuted across coarser groups—the exchangeability hypothesis under test—and the critical value is the $1 - \alpha$ quantile of $\hat{\rho}$ over 400 permutations at the observed group count and balance, $\alpha = 0.05$. A fixed threshold instead controls the null *mean*, not its *tail*: at a fixed 0.2 the measured null size was 0.17, so the rule halted on about one correct design in six and any clean-path pass was a property of the seed. Because specificity is a rate we measure it as one: over 450 clean evaluations of this rule alone (150 seeds $\times$ three environments) it halts 19 times, a rate of 0.042 with Wilson [10] interval $[0.027, 0.065]$—consistent with the nominal 0.05, not better—and it detects the injected fault on 150 of 150 paths. Finally the rule *declines to judge* below four coarser groups or eight units, returning INDETERMINATE and raising nothing: an explicit abstention, not a pass, though no condition here is small enough to exercise it.

Mutation canaries. Six channels are probed, separating two easily conflated questions: whether a channel is *guarded*, and whether the probe mutation was *effective*—an inert mutation reads as an undetectable leak, which L3.2 catches. *Provenance and seeds.* Seeds live in three disjoint namespaces—burned, debug, evaluation—and a seed whose outcomes have been inspected is burned for good. Compared conditions draw one seed per invariant cell, with pairing asserted on the realised arrays rather than inferred from the derivation rule.

IV. Curated Regression Evaluation

A. Design

Two pipelines serve as minimal deterministic fixtures. Case A exercises a retrospective-label chain: held state, visible-pass scheduling, frozen registry, later reference construction, closure. Case B exercises a learning and gating chain: physics baseline, optional learned branch, validation selection, frozen model/scaler/gate state, later deployment. Neither carries a physical claim.

The suite contains 17 *curated* fault classes, drawn from the defects that motivated the rules. Each is injected in three environments differing in pseudo-random generator family, giving 51 injected cells, plus one clean reference path per

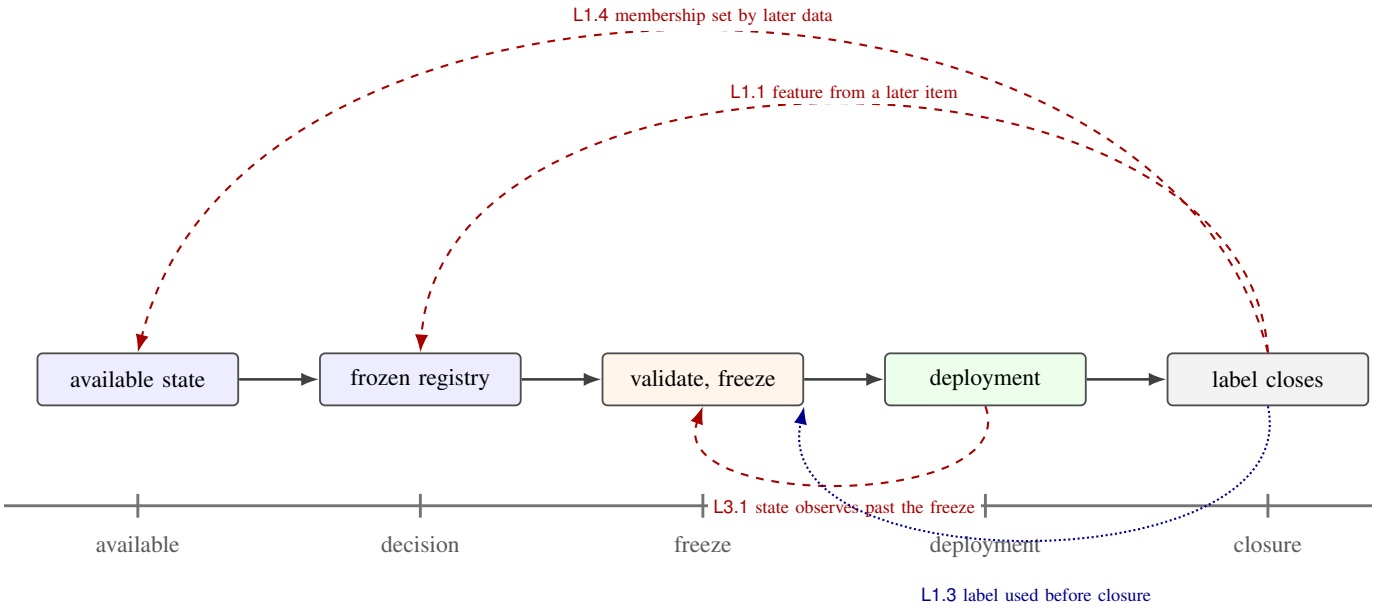

Fig. 1. The deployment-causality contract; axis ticks mark availability, the decision instant t_d, the freeze, deployment and closure t_c. Solid paths are permitted: published state held before t_d populates a registry whose rows are hashed before any label exists, model and gate are fixed at the freeze, deployment reads frozen state only, labels close retrospectively. The three red *dashed* paths are the point of this paper—each is *chronologically consistent*, so an ordering check cannot express it. The blue *dotted* path (L1.3) is the contrast case: one of the two classes ordering already catches.

TABLE I

THE NINETEEN RULES AND WHAT IS DEMONSTRATED FOR EACH. **CLEAN + RED** = A PASSING CLEAN-PATH TEST *and* A BROKEN FIXTURE ON WHICH THE RULE FAILS; **CLEAN ONLY** = IMPLEMENTED AND CLEAN-PATH-EXERCISED BUT TRIPPED BY NO FAULT HERE, WHICH BY OUR OWN STANDARD IS NOT YET EVIDENCE IT CAN FAIL. †L4.7 ALSO ABSTAINS BELOW ITS DECLARED MINIMUM GROUP STRUCTURE.

Rule	Protected object	Evidence	Rule	Protected object	Evidence
L1.1	feature availability	clean + red	L3.2	canary effectiveness	clean + red
L1.2	availability clock	clean + red	L3.3	negative control	clean + red
L1.3	label closure	clean + red	L4.1	fold chronology	clean + red
L1.4	row membership	clean + red	L4.2	paired randomness	clean + red
L1.5	comparison boundary	clean + red	L4.3	repeated measures	clean + red
L2.1	sampling geometry	clean + red	L4.4	seed hygiene	clean + red
L2.2	physical scale	clean only	L4.5	provenance hashes	clean only
L2.3	hidden state	clean only	L4.6	provenance completeness	clean + red
L2.4	declared relation	clean + red	L4.7	statistical unit	clean + red†
L3.1	state channels	clean + red			

environment reported separately. Three denominators are easily conflated, so we fix them: 19 *rules*, 17 fault *classes*, 18 *conditions* per environment. Rules and classes are not in bijection—one rule can be violated several ways, and some are represented by no fault here.

B. Results

Fig. 2 gives the coverage matrix, Fig. 3 the case studies of §V. Every number below comes from one artifact, `final_summary.json`, derived from the committed matrix result.

The chronological baseline is *implemented and executed*, not reasoned about: three checks assert that fold time-extents are ordered and disjoint, that no row sits in a fold whose span it does not belong to, and that no label used at a decision point postdates it. It runs in the same sweep, over the same fixtures and environments. It is not a competing detector suite: chronological splitting is a *protocol property* of limited scope, so the comparison measures how much of this fault space that property covers, not tool against tool. Measured, it detects $2/17$ classes—invalid label-closure timing and a violated fold order, precisely the two that *are* ordering properties—and fires on no clean path; the remaining fifteen lie outside what ordering can express. The contract detects $17/17$, identically in all three environments. We report 17 distinct computations rather than 51 as the coverage denominator: the environments differ only in generator family, and the object handed to the detector is bit-identical across them for fourteen of the seventeen classes. Only the repeated-measures, fold-order and statistical-unit faults actually move.

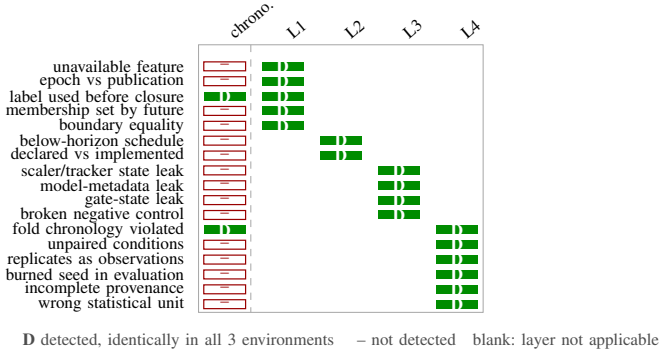

Fig. 2. Curated regression coverage over the 17 fault classes, grouped by detecting layer. Chronological protocol checks (left column) detect $2/17$—exactly the two classes that are ordering properties—while contract layers L1–L4 detect $17/17$, identically in all three environments, which differ in pseudo-random generator family and are not independent satellite systems or populations. The matrix measures represented-fault regression coverage; it does not estimate completeness on unseen faults.

The $51/51$ figure is therefore a *determinism* statement—every cell agrees—not seventeen results times three.

Clean reference paths were accepted: no rule fired on any of the three at the committed seed, and clean verdicts were identical across environments. For the one rule with a stochastic reference distribution the single clean run is not the evidence—the measured 0.042 false-halt rate is. Sixteen of the nineteen rules are demonstrated red by a fault fixture; three (L2.2, L2.3, L4.5) are exercised only on the clean path. Three faults additionally raise L3.2: an unguarded state channel also renders the mutation aimed at it inert, a true positive rather than noise. The full sweep is 18 conditions per environment over both pipelines and three environments. It runs in under 2 s on one core of a commodity laptop, under 30 ms per condition, dominated by L4.7's permutation null. With `numpy` as the only dependency it is affordable as a per-commit gate rather than a periodic audit.

One clean-path false positive was not designed: the first run reported three, all L4.4, one per environment—caused by the fixture, not the detector, since the clean pipeline declared it was about to execute a seed still in its own evaluation namespace. We repaired the fixture, changed no detector, and preserved both records. The sequence matters: adjusting the rule instead would have bought the clean-path rate by weakening the contract.

V. Case Studies

Both case studies are pipelines from a *stopped* research programme, built with chronological splitting and passed by it. They are *retrospective accounts*: both predate the contract released here, so we report which rule *names* each defect rather than a halt we executed, and no artifact survives for the second. Three of that programme's own checks were later found incapable of failing—one compared two arrays built from the same object, one pinned the parameter whose influence it tested, one attached no threshold—which is where the red-fixture requirement comes from. They are also not independent of Fig. 2: the fixtures were modelled on these pipelines and the fault classes derived from their defects. No model-performance quantity is reported.

A. Retrospective-label pipeline

A system holds a stale orbital element set and predicts open-loop; labels come from a later, better-determined element set in the same catalogue; folds are split chronologically by reference time. Three defects were present. A feature was the interval between the held element's *epoch* and the *reference* element's epoch—a difference of two past timestamps, hence ordering-consistent, but containing an item the system had not been sent (L1.1). Row membership depended on the future catalogue, since a transmission entered the dataset only if a qualifying later element was published (L1.4). Label availability was coupled to the covariate under study: a publication outage simultaneously makes the held element stale and removes the elements needed to label that staleness. An intermediate version also scheduled on a clock grid rather than inside predicted-visible geometry (L2.1).

Ordering sees none of these, because none violates ordering. A reported improvement therefore could not be attributed to information the system would hold; under the third defect the comparison is not merely biased but undetermined, the baseline being less tightly pinned than the effect sought.

B. Learning and gating pipeline

A physics baseline, an optional learned branch, selection on a chronologically prior validation window, an admission decision frozen before the deployment window, and fallback to the baseline. The feature tensor was clean and verified so by perturbation. Future information nevertheless entered through a non-feature state channel: a stateful tracker kept advancing at refresh events inside the deployment window, so the learner observed outcomes after its own freeze (L3.1). The negative control separately exposed undeclared deterministic signal—a rate frozen at one epoch while the reference keeps evolving leaves a residual that is a deterministic function of the covariate, hence learnable, with the injected effect at exactly zero (L3.3). And compared conditions failed to share a realisation, the condition label having entered the seed derivation (L4.2).

A row-level split governs rows and cannot constrain model state: the tracker's update was chronologically forward at every step and still wrong, because it crossed the freeze.

VI. Threats to Validity, Limitations and Conclusion

The faults and the rules share an origin: both were developed in one programme, mostly by the same hand, so $17/17$ measures detector *reachability*—a fixture written to trip rule X trips rule X—not mutation adequacy [7], and hand-authored mutants are known to overstate suite strength [11]. Only one fault was specified before any detector for it existed and left untouched afterwards, far too few to support a claim about faults the suite lacks, so we make none. What it

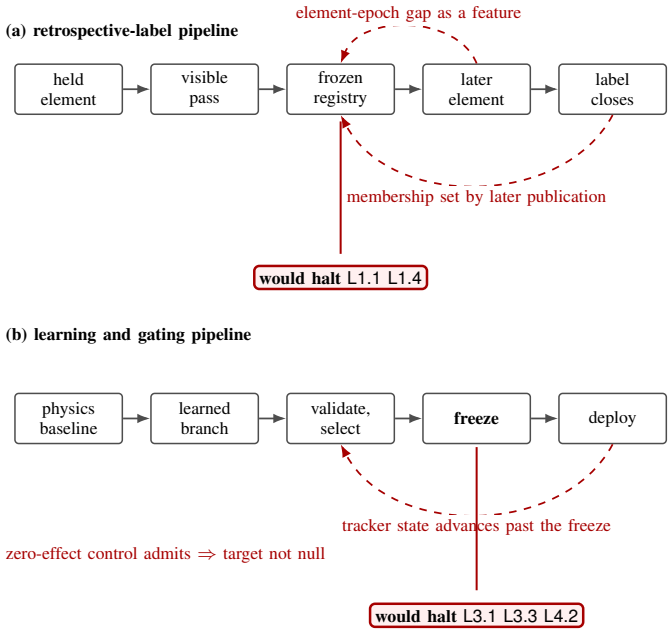

Fig. 3. The two case studies and the rule naming each defect. **Retrospective accounts, not executed runs**: both pipelines predate the released contract, hence *would halt*, and no artifact survives for (b). Satellite objects, left to right in (a): element publication versus epoch, visible-pass scheduling, the frozen registry fixing row membership, later catalogue publication, label closure; in (b) the physics baseline, the learned branch, validation selection, the frozen model/gate/tracker state, per-pass deployment. Dashed arrows are the defects, each chronologically consistent. No model-performance quantity is reported.

establishes is regression coverage of *represented* faults: these violations, once fixed, cannot silently return. Nor is the ratio a sensitivity—the denominator is curated, not a sample from a natural fault distribution.

Injection is at contract-input level, and the fixtures are check-scoped. Each fault selects a pre-shaped input rather than mutating pipeline source, and the fixtures hand every rule its own input attribute: of the seventeen mutated objects only six reach more than one consumer. For the other eleven the mutated object is consumed by its own detector alone, so those conditions show a rule firing when handed a broken input rather than catching a defect in a working pipeline. This is the sharpest limit on the evaluation, and why $17/17$ is stated as regression coverage. L2.2, L2.3 and L4.5 fire in no condition at all, which by our own standard is not evidence they *can* fail. And L4.3 still decides by a fixed correlation threshold gated on a self-reported aggregation flag—the uncalibrated construction we corrected in L4.7. Its null size depends on group structure: 0.00 at this fixture's ninety-six groups, 0.17 at eight groups of three. We disclose it rather than repair it, because editing a detector after its outcome is known is precisely what voided the standing of L4.7's own result.

Two objects the contract names but does not check. Covariate-coupled label availability is called the harder failure in the threat model, yet no rule detects it. It was found by an *ad hoc* thresholded diagnostic—a standardised mean difference between labelled and censored rows on pre-decision covariates, which fired on the study covariate in the stopped pipeline of §V and is recorded in the archived audit—never promoted to a rule, so it has no identifier and no red fixture. And t_d itself is declared, not checked: every L1 rule is conditional on it, but under periodic provisioning the defensible t_d is the refresh instant rather than the transmission instant, and declaring the latter makes every availability check pass while admitting up to one refresh interval of unheld state. L1.2 enforces a catalogue clock, which is necessary and not sufficient.

The axes of variation are narrow. Both fixtures come from this programme, so reuse across independent projects is argued rather than demonstrated, and the environments differ in generator family alone. No radio-frequency, packet-level, deployed-system or learned-method result is established here. Nor is the contract complete: it covers six protected objects we found necessary. Fault diversity in particular matters, and a regression suite protects exactly what it represents: showing that a rule catches defects nobody anticipated needs faults authored independently of the detectors and injected at program level.

What the evidence supports is narrow and, we think, useful. Chronological splitting addresses ordering and only ordering, and the fifteen classes it misses here are ordinary consequences of two clocks, a publisher-controlled label, predicted-visible geometry, pass-level dependence, or state outliving a freeze. An executable contract catches them cheaply enough to run on every commit, and names the obligation broken rather than reporting that something failed. Applying the same machinery where labels come from measurement would remove the covariate-coupled censoring this contract cannot yet detect.

References

[1] X. Lin, S. Rommer, S. Euler *et al.*, "5G from space: An overview of 3GPP non-terrestrial networks," *IEEE Communications Standards Magazine*, vol. 5, no. 4, pp. 147–153, 2021.

[2] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012.

[3] D. A. Vallado, P. Crawford, R. Hujsak, and T. S. Kelso, "Revisiting spacetrack report #3," *AIAA/AAS Astrodynamics Specialist Conf.*, 2006.

[4] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, 2012.

[5] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, 2023.

[6] D. Sculley, G. Holt, D. Golovin *et al.*, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems*, 2015.

[7] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Trans. Softw. Eng.*, vol. 37, no. 5, pp. 649–678, 2011.

[8] T. Gebru, J. Morgenstern, B. Vecchione *et al.*, "Datasheets for datasets," *Commun. ACM*, vol. 64, no. 12, pp. 86–92, 2021.

[9] P. E. Shrout and J. L. Fleiss, "Intraclass correlations: Uses in assessing rater reliability," *Psychological Bulletin*, vol. 86, no. 2, pp. 420–428, 1979.

[10] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[11] R. Just, D. Jalali, L. Inozemtseva *et al.*, "Are mutants a valid substitute for real faults in software testing?" in *Proc. ACM SIGSOFT Symp. Found. Softw. Eng. (FSE)*, 2014, pp. 654–665.